Answer: React Components, Props, and useState
1. Components
Components are the independent and reusable building blocks of a React application. They work in isolation and return HTML-like syntax (JSX) to define what should appear on the screen.

2. Props (Properties)
Props are read-only inputs passed to a component. They allow data to be transferred from a parent component to a child component, similar to function arguments.

3. useState
useState is a React Hook that lets you add state management to functional components. It returns an array containing the current state value and a function to update that value, triggering a re-render when the state changes.

Different Types of Components with Examples
There are two primary ways to define components in React:

1. Functional Components
These are simple JavaScript functions that accept props as an argument and return JSX. With the introduction of Hooks (like useState), functional components are now the standard way of writing React code.

Example:

jsx
```
import React, { useState } from 'react';

// Functional Component
function Greeting(props) {
// Using useState Hook
const [isLoggedIn, setIsLoggedIn] = useState(false);

return (
<div>
<h1>{props.title}</h1>
<p>Status: {isLoggedIn ? 'Logged In' : 'Logged Out'}</p>
<button onClick={() => setIsLoggedIn(!isLoggedIn)}>
Toggle Status
</button>
</div>
);
}

// Usage in a parent component
function App() {
return <Greeting title="Welcome to the App!" />;
}

export default App;
```
2. Class Components
These are more traditional ES6 classes that extend React.Component. They must contain a render() method that returns JSX. State is managed using this.state and updated with this.setState().

Example:

jsx

```
import React, { Component } from 'react';

// Class Component
class Greeting extends Component {
// Constructor to initialize state
constructor(props) {
super(props);
this.state = {
isLoggedIn: false
};
}

toggleLogin = () => {
this.setState(prevState => ({
isLoggedIn: !prevState.isLoggedIn
}));
};

// Render method
render() {
return (
<div>
<h1>{this.props.title}</h1>
<p>Status: {this.state.isLoggedIn ? 'Logged In' : 'Logged Out'}</p>
<button onClick={this.toggleLogin}>
Toggle Status
</button>
</div>
);
}
}

// Usage in a parent component
class App extends Component {
render() {
return <Greeting title="Welcome to the App!" />;
}
}

export default App;
```